

Flow:

Image $\rightarrow$ Filters $\rightarrow$ Feature Maps $\rightarrow$ Pooling $\rightarrow$ Deeper $\rightarrow$ Output Features

Page 1

CNN Feature Abstraction -

1) CNN architecture -

- Layers: Input $\rightarrow$ Hidden (Conv + Pool) $\rightarrow$ Fully Connected $\rightarrow$ Output
- Uses convolution + pooling + activation
- Learns complex patterns with many parameters

2) Convolution Basics -

- Combines input with filters/kernels $\rightarrow$ feature maps
- Feature Map = Input $\ast$ Kernel
- Detects edges, textures, shapes

3) Filters/Kernels -

- Filters = weights + bias (shared across image)
- Each filter extracts different feature
- Multiple filters $\rightarrow$ multiple feature maps

4) Convolutional Layer -

- Input: Image/feature maps
- Output: feature maps (activation maps)
- Partial connectivity (not fully connected)

5) Feature Extraction Process -

- Input $\rightarrow$ Convolution $\rightarrow$ Activation (ReLU) $\rightarrow$ Feature Map
- Deeper layers $\rightarrow$ more complex features

6) Pooling -

- Reduces spatial size (dimensionality)
- Helps in efficiency + generalization

7) Key Principles -

- Parameter sharing: same filter across image
- Local connectivity: neuron connects to small regions

8) Convolution Operation -

- Slide kernel over image
- Compute weighted sum of neighborhood
- Replace pixel with computed value

9) Benefits -

- Reduces parameters
- Faster computation
- Captures hierarchical features

10) Padding -

- Add extra pixels (usually 0s) around input

- Types:

- Padding = 0 $\rightarrow$ no padding

- Padding = 1 $\rightarrow$ 1-pixel border

- Purpose:

- Preserve spatial size

- Avoid info loss at edges

11) Stride -

- Step size of filter movement

- Stride = 1 $\rightarrow$ moves 1 pixel

- Stride = 2 $\rightarrow$ skips pixels $\rightarrow$ smaller output

- Larger stride $\rightarrow$ smaller output size

12) Pooling -

- Reduces spatial dimensions

- Types:

- Max Pooling $\rightarrow$ picks max value

- Average Pooling $\rightarrow$ computes average

- Benefits:

- Reduce computation

- Prevent overfitting

- Highlight important features

13) Convolution Process -

- Slide filter over image

- Compute weighted sum of local region

- Generate feature map

- Learn patterns (edges, textures)

14) Feature Map Formula -

$$\text{Feature map} = \text{Input Image} * \text{Kernel}$$

15) Key CNN Concepts -

- Parameter sharing $\rightarrow$ same filter across image
- Local connectivity $\rightarrow$ neuron sees small region
- Reduces parameters $\rightarrow$ faster learning

16) Layer wise Processing -

- Input $\rightarrow$ Conv ~~filter~~ $\rightarrow$ Activation (ReLU) $\rightarrow$ Pooling
- Deepen layers $\rightarrow$ more abstract features

17) Fully Connected Layer -

- Connects all neurons from previous layer
- Produces final feature vector
- Used for classification

18) Softmax (Output layer) -

- Converts outputs to probabilities : $f(x_i) = \frac{e^{x_i}}{\sum e^{x_i}}$
- Used for multi class classification

19) Working Intuition -

- Sliding window detects patterns
- Same filter used everywhere
- Builds new representation layer by layer

Hyperparameter Tuning -

- Selecting optimal hyperparameters (set before training)
- Controls learning process behavior
- Prevents overfitting / underfitting
- Improved generalization & accuracy
- Treated as a search space problem
- Techniques

- Grid Search CV

- Randomized Search CV

Grid = Exhaustive, accurate, slow
Random = Faster, scalable, approximate

Date |

Page |

Grid Search CV -

- Brute force search over all combinations

- Steps:

- (i) Define grid of values
- (ii) Train for each combination
- (iii) Evaluate via cross validation
- (iv) Select best score

- Total models = (values of param 1 $\times$ param 2 $\times$...)

eg - 5(C) $\times$ 4(α) = 20 models

- Cons: Slow, computationally expensive

$$\text{Total combinations} = N = n_1 \times n_2 \times \dots \times n_k$$

(n = values)

n = values
per
hyper
parameter

Randomized Search CV -

- ~~Simple~~ Samples random combinations of all

- Process:

- (i) Try random sets
- (ii) Evaluate performance
- (iii) Select best

- Uses cross-validation

- Pros: Faster, efficient for large search space

- Cons: May miss optimal combination

§

Advantages of Hyperparameter tuning -

- Improved model performance
- Reduced overfitting & underfitting
- Enhanced model generalizability
- Optimized resource utilization
- Improved model interpretability

Challenges -

- (i) Dealing with high dimensional hyperparameter spaces
- (ii) Developing adaptive hyperparameter tuning methods

LR $\rightarrow$ speed of learning
Batch $\rightarrow$ data per step
Weight decay $\rightarrow$ prevent overfitting

Date |

Page |

Hyperparameters -

- 1) Learning Rate (LR) $\rightarrow$ step size of updates
- 2) Epochs $\rightarrow$ full pass over dataset
- 3) Batch size $\rightarrow$ samples per update
- 4) Optimizer $\rightarrow$ update rule (GD, SGD, Adam, etc)
- 5) Weight Decay (λ) $\rightarrow$ regularization

Learning Rate (LR) -

- 1) Too small $\rightarrow$ slow learning/underfitting
- 2) Too large $\rightarrow$ divergence
- 3) Use LR range test to find bounds
- 4) Cyclical LR:
 $\hookrightarrow$ varies between $\text{min-LR} \leftrightarrow \text{max-LR}$

LR Range Test -

- 1) Increases LR linearly during training
- 2) Pick range where loss decreases
- 3) Rule:
 $\text{min-LR} = \text{max-LR} / 10$

Batch size -

- 1) Limited by GPU memory
- 2) Total batch = batch per GPU $\times$ no. of GPU
- 3) Small batch $\rightarrow$ more regularization
- 4) Large batch $\rightarrow$ faster training, less noise

Weight decay -

- 1) L2 Regularization
- 2) Prevents overfitting
- 3) Update rule: ~~$w = w - \eta \nabla L$~~ $w = w - \eta \nabla L - \lambda w$
- 4) $\lambda \in (0, 1)$, usually constant

Epochs $\rightarrow$ how many times I learn
Optimizer $\rightarrow$ how to update

Date |

Page |

Epochs -

- 1) 1 epoch = full dataset pass
- 2) Steps per epoch:
$$\text{steps} = \frac{\text{total samples}}{\text{batch-size}}$$
- 3) eg:
1000 samples, batch = 100 $\rightarrow$ 10 steps / epoch
10 epochs $\rightarrow$ 100 steps total
- 4) Too few $\rightarrow$ underfitting
- 5) Too many $\rightarrow$ overfitting

Optimizers -

- 1) Gradient Descent: full dataset update
- 2) SGD: per sample update
- 3) Adagrad: adaptive LR (per parameter)
- 4) Momentum: accelerates convergence
- 5) Adam: combines Momentum + RMS Prop
- 6) RMS Prop: handles non stationary data

Gradient Descent -

- 1) Optimization to minimize loss $J(w)$
- 2) Goal: find optimal weights
- 3) Update rule:
$$\begin{aligned} W &= W - \eta \cdot dW \\ b &= b - \eta \cdot db \end{aligned} \quad (\eta = \text{learning rate})$$
- 4) Working:
 - Start with random initialization
 - Iteratively update weights using gradients
 - Moves towards minimum loss (global / local)
- 5) LR effect:
 - Small LR $\rightarrow$ slow convergence
 - Large LR $\rightarrow$ overshoot / diverge
 - Optimal LR $\rightarrow$ fast stable convergence

GD $\rightarrow$ basic optimization

LR $\rightarrow$ step wise

Momentum $\rightarrow$ smooth + faster path to minima

Date: / /

Page: /

6) Local vs Global minimum

└ GD may get stuck in local minima

└ Initialization affects final solution

7) Oscillation Problem

└ High LR $\rightarrow$ zig-zag / oscillations

└ Slows convergence

8) ~~Mom~~

Momentum (Improvement over GD) -

1) Uses exponentially weighted average of gradients

2) Velocity

$$\text{└ } V_{dw} = \beta V_{dw} + (1 - \beta) dw$$

$$\text{└ } V_{db} = \beta V_{db} + (1 - \beta) db$$

3) Update

$$\text{└ } w = w - \eta \cdot V_{dw}$$

$$\text{└ } b = b - \eta \cdot V_{db}$$

$$(\beta \in [0, 1])$$

4) Benefits

└ Reduced oscillations

└ Faster convergence in correct direction

└ Helps escape shallow local minima

Types of Gradient Descent -

1) Batch GD: uses full dataset $\rightarrow$ stable

2) SGD: one sample $\rightarrow$ fast but noisy

3) Mini batch: small batches $\rightarrow$ best tradeoff

Loss Function -

└ Measures prediction error

└ eg:

$$Loss = (y - \hat{y})^2$$

Momentum Equations -

(i) Velocity update:

$$V_i = \beta V_{i-1} + (1 - \beta) \nabla L(\theta)$$

(ii) Parameter update:

$$\theta = \theta - V_i$$

(iii) β (momentum) $\in [0, 1]$

Need for Momentum -

- Loss surface is non convex
(multiple minima, saddle points)

- SGD may oscillate or get stuck

- Momentum $\rightarrow$ smooth + faster updates convergence

Effect of Momentum -

- Reduces vertical oscillations

- Increases horizontal movement (faster progress)

- Helps reach minima in fewer iterations

Mini batch GD -

- Processes b samples per iteration

- Balance between speed & stability

- Adds hyperparameter: batch size (b)

Ada Grad -

1) Adaptive learning rate per parameter

2) Works well for sparse data

AdaGrad Update rule:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{V_t + \epsilon}} \cdot \text{gradient}$$

$$V_t = \sum (\text{past gradients}^2)$$

$\epsilon = \text{small constant}$

3) Key Insights

- Large gradients $\rightarrow$ LR decreases

- Smaller gradients $\rightarrow$ LR stays higher

- Focused on important sparse features

4) Limitations

- LR keeps decreasing $\rightarrow$ very slow convergence

- Improved by: RMSProp, Adam

RMSProp $\rightarrow$ fixed LR decay
Adam $\rightarrow$ best default optimizer

Date |

Page |

RMSProp -

- 1) Fixed ~~and~~ AdaGrad issue: learning rate decays too much
- 2) Uses moving average of squared gradients
- 3) Focuses more on recent gradients
- 4) Formula:

$$v_t = \beta v_{t-1} + (1-\beta) g_t^2$$

$$\theta = \theta - \frac{\eta}{\sqrt{v_t} + \epsilon} \cdot g_t$$

$\beta \approx 0.9 - 0.95$, $\epsilon = \text{small constant}$

5) Why?

- Prevents vanishing learning rate
- Works well for non convex loss surfaces
- More stable than AdaGrad

Adam Optimizer -

- 1) Combines Momentum + RMSProp
- 2) Uses adaptive learning rate per parameter

3) Equations:

(a) First moment (mean):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

(b) Second moment (variance):

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

(c) Update rule:

$$\theta = \theta - \frac{\eta}{\sqrt{v_t} + \epsilon} \cdot m_t$$

4) Advantages

- Faster convergence
- Handles sparse gradients
- Works well with minimal tuning
- Stable across different datasets

~~Disadvantages~~

- ~~- Adaptive LR scaling~~
- ~~- Momentum smoothing~~
- ~~- Bias correction~~

5) Key Features

- Adaptive LR scaling
- Momentum smoothing
- Bias correction

Dropout $\rightarrow$ drop neurons
Drop Connect $\rightarrow$ drop weights
Batch Norm $\rightarrow$ normalize inputs

Date |

Page |

Regularization -

- 1) Prevents overfitting
- 2) Improves generalization
- 3) Adds penalty term to loss
- 4) Techniques:

- Dropout
- Drop Connect
- Batch Normalization

Dropout -

- Randomly deactivates neurons during training
- Prevents over-reliance on specific neurons
- Acts like ensemble learning

Drop Connect -

- Randomly sets weights to zero
- Drops connections instead of units

Batch Normalization -

- Normalizes inputs: $x' = \frac{x - \mu}{\sigma}$

- Reduces internal covariate shift

- Adds learnable ~~param~~ params: γ (scale), β (shift)

- Improves training speed & stability